File Secure Drop

Implementasi aplikasi pengiriman dokumen menggunakan pertukaran kunci Diffie-Hellman dan enkripsi-dekripsi ElGamal dalam satu studi kasus.

Identitas kelompok

Kelompok / kelas

Nama anggota dan NRP (satu anggota per baris)

Studi kasus dan tujuan

Bob mengirim dokumen kepada Alice. Bob memilih file dan penerima, kemudian sistem menyepakati rahasia sesi, menyandi isi file, dan menyimpan kiriman. Alice membuka kotak masuk dan mengunduh file. Sistem memeriksa integritas serta mendekripsinya secara otomatis.

Antarmuka hanya menampilkan fitur pengiriman file. Rumus, kunci, ciphertext, dan nilai perantara tersedia melalui fungsi serta komentar di kode untuk pemeriksaan tugas.

Lingkungan dan cara menjalankan

Bagian	Implementasi
Antarmuka	HTML, CSS, JavaScript murni; tanpa library, framework atau CDN.
Server dan penyimpanan	Python 3.10+ standard library, HTTP lokal, SQLite.
Kriptografi	DH, ElGamal, modular exponentiation, invers, SHA-256 dan HMAC ditulis mandiri.
Menjalankan	Ekstrak ZIP, klik JALANKAN.bat, atau python server.py --open. Buka http://127.0.0.1:8772.

Penerapan algoritma

1. Pembangkitan kunci dan pertukaran DH

Setiap akun mempunyai pasangan kunci ElGamal: privat x dan publik y. Untuk setiap file, kedua pihak membuat a dan b acak baru, bertukar A dan B, lalu menghitung K masing-masing. Fungsi diffie_hellman menolak sesi jika hasil kedua pihak berbeda.

```
y = g^x mod p
A = g^a mod p; B = g^b mod p
K_pengirim = B^a mod p; K_penerima = A^b mod p
```

Parameter aplikasi: p = 2305843009213693951 (2^61 - 1), g = 37. validate_parameters membuktikan prima Mersenne dengan Lucas-Lehmer dan memeriksa akar primitif melalui faktor prima p-1. Eksponen acak memakai sumber angka secrets bawaan Python, bukan algoritma enkripsi siap pakai.

2. Enkripsi file langsung dengan ElGamal

Catatan dan jenis file dimasukkan ke header, lalu digabungkan dengan byte file. Payload dipecah menjadi blok 7 byte. Nilai tiap blok kurang dari 2^56 dan pasti lebih kecil dari p. Setiap blok memakai k acak berbeda dalam satu file.

```
c1 = g^k mod p
s = y^k mod p
c2 = m * s mod p
```

Setiap c1 dan c2 disimpan sebagai bilangan 8 byte. Panjang payload disimpan agar blok terakhir dan byte nol di depan dapat dipulihkan dengan tepat. Isi file tidak diganti dengan enkripsi simetri: setiap blok benar-benar dienkripsi ElGamal.

3. Integritas dan dekripsi di sisi penerima

K DH diolah menjadi kunci HMAC melalui SHA-256 mandiri. HMAC melindungi metadata dan seluruh ciphertext. Penerima menghitung K dari A^b dan memeriksa HMAC sebelum mendekripsi. Tambahan ini membuat hasil DH dipakai dalam alur pengiriman, serta mendeteksi perubahan kiriman.

```
s = c1^x mod p
r = invers(s, p)
m = c2 * r mod p
```

Invers dihitung dengan Euclid diperluas. File hasil pemulihan diperiksa lagi menggunakan ukuran dan SHA-256 file asli. Kunci privat tidak dikirim ke antarmuka. SHA-256/HMAC menjadi tambahan integritas; keduanya bukan pengganti DH atau ElGamal.

Pengujian dan hasil

Pengujian otomatis dijalankan dengan python -m unittest discover -s tests -v. Seluruh 12 pengujian lulus. Pengujian browser terpisah memeriksa alur pengiriman sungguhan melalui server lokal.

Pengujian	Hasil yang diverifikasi
Angka DH dari PPT slide 11	p=251, g=6, a=124, b=158: A=209, B=197, K kedua pihak=79.
ElGamal PPT slide 15-20	p=2273, g=3, x=243, y=461; HALO: 0700/1114; ciphertext (1439,74)/(1220,1682); invers 1791/1125.
File biner dan encoding	Seluruh nilai byte 0-255, Unicode, file kosong, byte nol, dan blok terakhir berhasil dipulihkan.
Oracle independen	Square-and-multiply dibandingkan dengan perpangkatan langsung. SHA-256 dan HMAC dibandingkan dengan oracle standard library hanya pada tes.
Keutuhan dan akses	Ciphertext/metadata yang diubah ditolak. Akun ketiga tidak dapat membuka kiriman. File database tidak dilayani HTTP.
Alur dua akun di browser	Bob mengirim file biner ke Alice, Alice reload dan unduh. Byte unduhan identik dengan asli; status Bob menjadi Sudah dibuka.

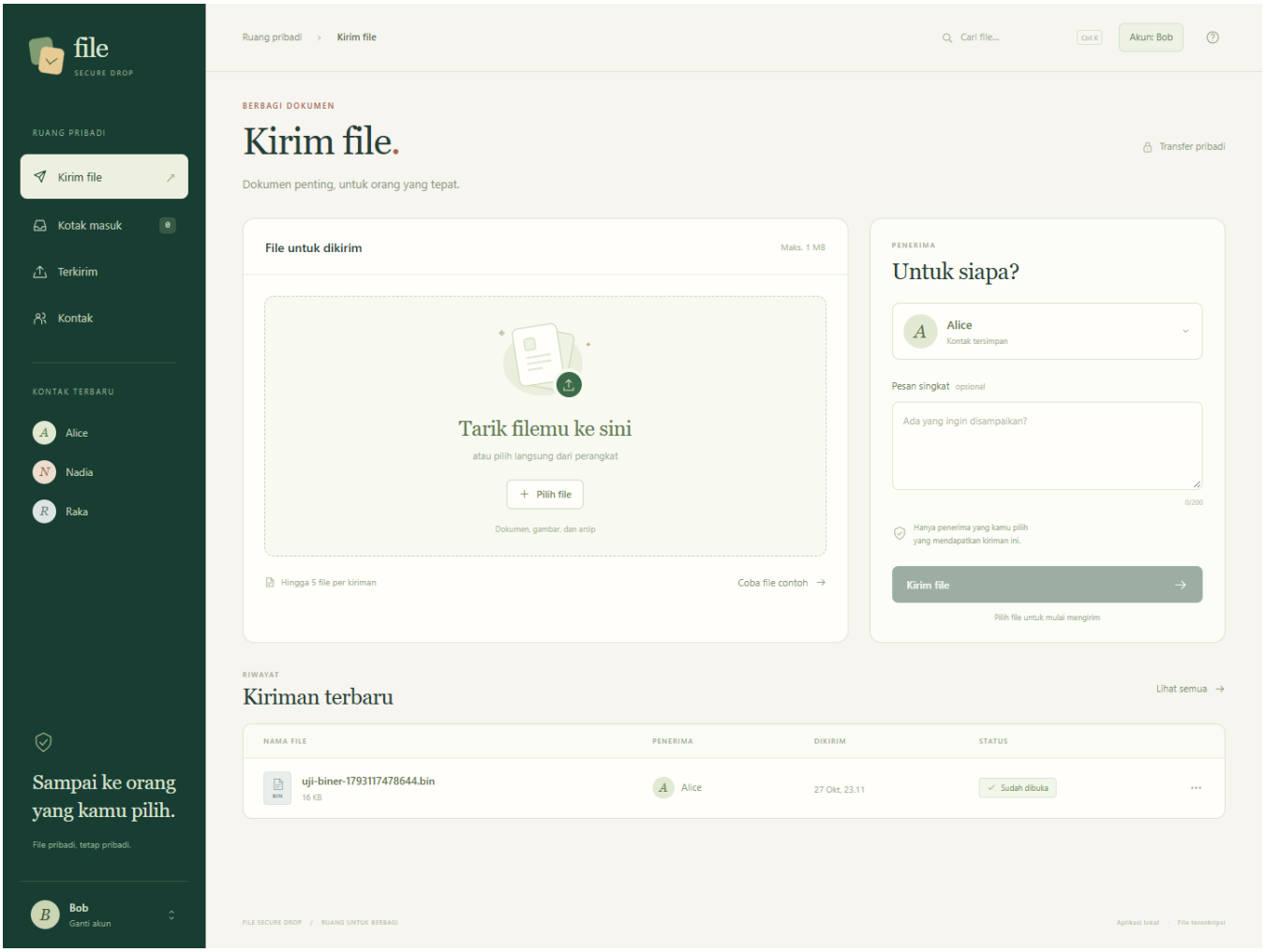
Peta fungsi untuk pemeriksaan

Lokasi	Fungsi utama
crypto_core.py	mod_pow, inverse, keygen, diffie_hellman, encrypt_number, decrypt_number, encrypt_bytes, decrypt_bytes, sha256, hmac_sha256.
server.py	make_record: seluruh tahap pengiriman; receive_record: verifikasi dan dekripsi; state_for: daftar kiriman per akun.
app.js	Pemilihan file, akun dan penerima, kirim ke API, daftar kiriman, dan unduh hasil dekripsi.

Bukti pengujian tersedia di tests/hasil-uji.txt dan tests/hasil-uji-browser.txt. Oracle hashlib/hmac hanya di tes; aplikasi tidak mengimpornya untuk kriptografi.

Penggunaan dan batasan

Demo: masuk sebagai Bob, pilih file maksimal 1 MB, pilih Alice, lalu Kirim file. Ganti akun lewat tombol Akun: Bob menjadi Alice. Buka Kotak masuk dan unduh kiriman. Dua tab/browser dapat dipakai dengan akun berbeda. File dan status tetap ada setelah reload atau server dimulai ulang.



Antarmuka menampilkan fungsi pengiriman, penerima, catatan, serta riwayat. Proses kriptografi berjalan otomatis di server.

Batas cakupan

Maksimal 5 file dengan total 1 MB per kiriman. Ini simulasi lokal untuk tugas: akun dipilih tanpa password, server mengelola kedua sisi dan kunci privat, serta hanya mendengarkan 127.0.0.1. Aplikasi bukan layanan end-to-end atau sistem produksi. Parameter 61 bit kecil untuk keamanan nyata; DH tanpa autentikasi identitas tidak mengatasi MITM. HTTP lokal tidak menyediakan TLS.

Referensi

PPT Pertemuan ke-9_El Gamal dan Diffie-Hellman, slide 8-20.

Menezes, van Oorschot, Vanstone. Handbook of Applied Cryptography, bagian 8.4 dan 12.6. https://cacr.uwaterloo.ca/hac/about/chap8.pdf

NIST, FIPS 180-4, Secure Hash Standard (SHA-256). https://csrc.nist.gov/pubs/fips/180-4/upd1/final

Krawczyk, Bellare, Canetti. RFC 2104, HMAC. https://www.rfc-editor.org/rfc/rfc2104